

Plan: Stream Isaac Sim in AR on Apple Vision Pro

Kuka Med-7 Surgical Planning — VisionProTeleop Integration

Date: March 2026
Project: Kuka Med-7 Surgical Planning Simulation
Library: [Improbable-AI/VisionProTeleop](#) (`avp_stream`)

Overview

Instead of streaming a 2D video feed from Isaac Sim to the headset, this plan uses the `avp_stream` library to render the entire simulation natively in **Apple RealityKit** on the Vision Pro. The USD stage is exported once as USDZ; thereafter only **6-DOF pose deltas** (~kilobytes per frame) are transmitted via WebRTC. RealityKit renders everything at native headset resolution with full depth and spatial audio.

Current vs New Stack

Component	Current	New
Hand tracking input	OpenXR device → Isaac Lab <code>OpenXRDevice</code>	<code>avp_stream</code> → Vision Pro ARKit
Scene rendering	Isaac Sim viewport (2D/3D window)	RealityKit on Vision Pro (native AR)
Hand data format	<code>dict</code> with flat arrays, snake_case keys	4×4 transform matrices, camelCase attrs
Point clouds (ROS)	USD Points prim (live in stage)	Automatically included in USDZ export
Probe / guide line prims	Already in USD stage	Automatically included in AR
Pinch gestures	OpenXR pinch threshold	<code>pinch_distance</code> float from <code>avp_stream</code>
Network transport	OpenXR local streaming	WebRTC (local IP or cross-network room code)

Prerequisites

Item	Action
Tracking Streamer VisionOS app	Install from App Store
avp_stream Python library	<code>pip install --upgrade avp_stream</code>
Vision Pro and workstation on same Wi-Fi	Required for local IP mode
Vision Pro IP address	Shown inside Tracking Streamer app

Step 1 — Install the Library

```
pip install --upgrade avp_stream
```

No other network configuration is required.

Step 2 — Replace Streamer Init in `teleop_med7_vr.py`

Remove the `OpenXRDevice` / `OpenXRDeviceCfg` setup.

Add the following near the top of `main()`, **after** `env.reset()`:

```
from avp_stream import VisionProStreamer

AVP_IP = "192.168.x.x" # Vision Pro IP shown in the Tracking Streamer app

streamer = VisionProStreamer(ip=AVP_IP, origin="sim")

streamer.configure_isaac(
    scene=env.scene,
    relative_to=[0, 0, 1.651, 0], # x, y, z (m), yaw (deg)
    include_ground=True,
    env_indices=[0], # stream only env 0
    force_reload=False, # skip re-export on warm restarts
)
streamer.start_webrtc()
```

`relative_to` explained

This places the simulation's world origin in your **physical room**:

- `z = 1.651` → puts `lbr_link_0` (world origin) at 5 ft 5 in above the Vision Pro's detected floor plane — matching the physical table height.
- `yaw = 0` → sim X-axis aligns with Vision Pro's X-axis. Adjust after first boot if the robot faces the wrong direction.

Step 3 — Replace Hand Data Reading Each Frame

Your current loop reads `rh["index_tip"]` from the OpenXR device.

Replace with `avp_stream`'s `TrackingData` object:

```
tracking = streamer.get_latest()
rh = tracking.right    # HandData: (27, 4, 4) world-frame joint transforms
lh = tracking.left

# Index tip position – replaces _joint_xyz(rh, "index_tip")
idx_tip = rh.indexTip[:3, 3]          # (x, y, z) in sim/lbr_link_0 frame
idx_base = rh.indexIntermediateBase[:3, 3]

# Pinch detection – replaces OpenXR pinch logic
pinch_r = rh.pinch_distance < 0.03    # True when thumb-index < 3 cm
pinch_l = lh.pinch_distance < 0.03
```

Because `origin="sim"` is set, all positions are **already in** `lbr_link_0` **coordinates**. No additional transform is needed.

Step 4 — Stream Pose Updates Each Frame

Add one line at the **end of the sim loop**, after `env.step()`:

```
streamer.update_sim()
```

This sends the current 6-DOF pose of every prim (robot joints, probe sphere, guide line, bone point clouds) to Vision Pro. Each update is a few kilobytes — not a rendered frame — so latency is minimal. Throttle to ~60 Hz if needed:

```
if step_count % 2 == 0:    # at 120 Hz physics → 60 Hz pose stream
    streamer.update_sim()
```

Step 5 — Point Clouds and Surgical Pointer in AR

`configure_isaac(env_indices=[0])` exports everything under `/World/envs/env_0/`. All surgical planning prims have been moved under this path so they appear in AR automatically.

Prims moved to `env_0`

Prim	Old path	New path
Femur point cloud	<code>/World/PointClouds/femur</code>	<code>/World/envs/env_0/PointClouds/femur</code>
Tibia point cloud	<code>/World/PointClouds/tibia</code>	<code>/World/envs/env_0/PointClouds/tibia</code>
Probe sphere	<code>/World/SurgicalPlan/Probe</code>	<code>/World/envs/env_0/SurgicalPlan/Probe</code>
Guide line cylinder	<code>/World/SurgicalPlan/GuideLine</code>	<code>/World/envs/env_0/SurgicalPlan/GuideLine</code>
Right hand stick	<code>/World/SurgicalPlan/HandStick/Right</code>	<code>/World/envs/env_0/SurgicalPlan/HandStick/Right</code>
Left hand stick	<code>/World/SurgicalPlan/HandStick/Left</code>	<code>/World/envs/env_0/SurgicalPlan/HandStick/Left</code>

Files modified: `med7_env.py` (point clouds), `teleop_med7_vr.py` (surgical pointer + hand stick).

When `update_pointcloud()` and `streamer.update_sim()` are called each frame, all prims update live in AR — point clouds, probe position, and guide line included.

Step 6 — Gesture Mapping

Replace your OpenXR gesture helpers with the `avp_stream` equivalents:

Current OpenXR gesture	<code>avp_stream</code> replacement
<code>pinch_l</code>	<code>lh.pinch_distance < 0.03</code>
<code>pinch_r</code>	<code>rh.pinch_distance < 0.03</code>

Current OpenXR gesture	avp_stream replacement
<code>_spread_thumb_pinky(lh)</code>	<code>np.linalg.norm(lh.thumbTip[:3,3] - lh.littleTip[:3,3])</code>
Wrist pose (hand proxy)	<code>rh.wrist</code> → 4x4 matrix, extract position + quaternion
<code>rh["index_tip"]</code>	<code>rh.indexTip[:3, 3]</code>
<code>rh["index_proximal"]</code>	<code>rh.indexIntermediateBase[:3, 3]</code>

Full joint name mapping (OpenXR → avp_stream)

OpenXR (snake_case)	avp_stream (camelCase)	Joint index
<code>wrist</code>	<code>wrist</code>	0
<code>index_tip</code>	<code>indexTip</code>	9
<code>index_proximal</code>	<code>indexKnuckle</code>	6
<code>index_intermediate</code>	<code>indexIntermediateBase</code>	7
<code>thumb_tip</code>	<code>thumbTip</code>	4
<code>little_tip</code>	<code>littleTip</code>	24
<code>middle_proximal</code>	<code>middleKnuckle</code>	11

Step 7 — Files That Do NOT Change

The following files require **zero modifications**:

- `med7_env_cfg.py` — scene config unchanged
- `med7_env.py` — `update_pointcloud()` / `update_tibia_pointcloud()` unchanged
- `med7_ros_pointcloud.py` — ROS point cloud pipeline unchanged
- `usd_pointcloud_viz.py` — USD Points update unchanged
- `mock_bone_publisher.py` — ROS bone publisher unchanged
- ROS publishing of `PoseStamped` on `/surgical_plan/probe_pose` — unchanged

Step 8 — Cross-Network Mode (Optional)

If the Vision Pro and workstation are on **different networks** (e.g., hospital Wi-Fi with firewall, remote access), use a room code instead of an IP:

```
# On Vision Pro: Tracking Streamer generates a room code, e.g. "ABC-1234"
streamer = VisionProStreamer(ip="ABC-1234", origin="sim")
```

Everything else stays the same. Latency will be slightly higher due to TURN relay routing.

Known Unknowns — Validate Before Implementation

#	Item	Risk	Resolution
1	Point cloud prim paths under <code>env_0</code> ?	~~Medium~~ RESOLVED	Moved to <code>/World/envs/env_0/PointClouds/</code> in <code>med7_env.py</code>
2	USDZ export size with bone USD files	Low	Use <code>force_reload=False</code> after first run
3	<code>update_sim()</code> at 120 Hz overwhelming WebRTC	Low	Throttle to 60 Hz with <code>step_count % 2</code>
4	<code>relative_to</code> z-offset calibration	Medium	Fine-tune after first boot in AR
5	OpenXR device still referenced in <code>med7_env_cfg.py</code>	Low	Remove <code>xr = OpenXRDeviceCfg(...)</code> from config

Implementation Order

1. `pip install --upgrade avp_stream`
2. Remove `OpenXRDeviceCfg` from `med7_env_cfg.py`
3. Add `VisionProStreamer` init + `configure_isaac` in `teleop_med7_vr.py`
4. Adapt hand data reading (OpenXR dict → `TrackingData` attributes)
5. Adapt gesture detection (pinch, spread)
6. Add `streamer.update_sim()` in the sim loop
7. Boot, verify point clouds appear in AR, calibrate `relative_to`

Expected Outcome

- Robot arm, hospital bed, bones, probe sphere, and guide line all visible in **AR overlay** on the physical operating table
- Hand tracking from Vision Pro drives the probe in real-time (sub-100 ms latency on Wi-Fi)

- Femur/tibia point clouds from ROS appear and update live in AR
 - Left pinch locking gestures work identically to current OpenXR setup
 - ROS surgical plan topics (`/surgical_plan/probe_pose` , `/surgical_plan/guide_line`) continue publishing when probe is locked
-

Generated from project: Kuka Med-7 Surgical Planning Simulation

Library reference: <https://github.com/Improbable-AI/VisionProTeleop>